

CSE 816 — Software Production Engineering
IIIT Bangalore

Stock Portfolio Manager

Final Project Report

Team Members

Name	Roll Number	Responsibilities
Chaitanya Nemade	MT2025032	Backend APIs, JWT Security, Finnhub Integration, Thymeleaf UI, Jenkins CI/CD, Kubernetes
Pankaj Deopa	MT2025081	Docker, Docker Compose, Ansible Playbooks & Vault, ELK Stack, Kibana Dashboards

May 2026

GitHub: github.com/3-pi-radians/stock-portfolio-manager

Table of Contents

1. Project Overview.....	3
1.1 Application Features.....	3
1.2 Microservices Architecture Overview.....	3
2. Technology Stack	5
2.1 Application Layer.....	5
2.2 DevOps Pipeline.....	5
3. Application Architecture	6
3.1 Service Breakdown.....	6
3.2 Source Code Structure	6
3.3 Data Model (ERD)	6
3.4 REST API Endpoints	7
4. Security Design.....	8
4.1 Authentication — Stateless JWT	8
4.2 Role-Based Access Control.....	8
4.3 Secrets Management	8
5. DevOps Pipeline — Detailed Breakdown	9
5.1 Version Control Strategy	9
5.2 Continuous Integration — Jenkins (4 Pipelines)	9
5.3 Containerisation — Docker.....	10
5.4 Configuration Management — Ansible (4 Roles)	10
5.5 Container Orchestration — Kubernetes with HPA	11
5.6 Monitoring — ELK Stack	11
6. End-to-End Pipeline Flow.....	12
7. Docker Hub Images	13
8. Advanced Features.....	14
8.1 Ansible Vault for Secrets	14
8.2 Kubernetes HPA.....	14
8.3 Microservices Architecture.....	14
8.4 Ansible Roles — Modular Design	14
8.5 Filebeat DaemonSet on Kubernetes.....	14

1. Project Overview

The Stock Portfolio Manager is a full-stack web application that enables users to securely register, manage isolated stock portfolios, and track live market prices in real time. The application is built on Java Spring Boot 3.2 using a microservices architecture, served via a Thymeleaf-driven UI through an API gateway.

The core focus of the project is the enterprise-grade DevOps pipeline encompassing containerisation, CI/CD automation, configuration management, container orchestration, and centralised observability.

1.1 Application Features

Feature	Description
User Registration & Login	Stateless JWT authentication stored in HTTP-only cookies
Portfolio Management	Create and manage multiple isolated stock portfolios per user
Holdings & Trading	Buy and sell stocks; track real-time current price and P&L
Live Market Data	Integration with Finnhub API for real-time stock prices
Transaction History	Full audit trail of every buy/sell operation
Admin Dashboard	Role-based access to monitor all users and activity
Auto-refresh	Stock prices refresh automatically every 60 seconds
Client-Side Validation	Form validation prevents malformed data submission

1.2 Microservices Architecture Overview

The application is built as three independent microservices communicating over a shared Docker network:

- auth-service (port 8081 → host 8200) — Handles registration, login, logout, and JWT token issuance
- app-service (port 8082 → host 8201) — Manages portfolios, holdings, market data, and admin functions
- api-gateway (port 8080 → host 8202) — Routes all requests, validates JWT tokens at the gateway level



Figure 1: Microservices Architecture — Complete Stack

2. Technology Stack

2.1 Application Layer

Layer	Technology	Version
Language	Java	21 (LTS)
Framework	Spring Boot	3.2.0
Security	Spring Security + JWT (jjwt)	0.11.5
Database	MySQL	8.0
ORM	Spring Data JPA / Hibernate	—
Frontend	Thymeleaf + HTML/CSS	—
Market Data API	Finnhub REST API	—
Build Tool	Apache Maven	3.8+
Gateway	Spring Cloud Gateway	2023.0.0

2.2 DevOps Pipeline

Stage	Tool	Purpose
Version Control	Git + GitHub	Source code management, branching strategy
CI/CD	Jenkins (4 pipelines)	Pipeline automation via GitHub Webhooks
Containerisation	Docker + Docker Compose	Multi-container application packaging
Configuration Mgmt.	Ansible + Ansible Vault	Automated deployment & secrets management
Orchestration	Kubernetes (K8s)	Container scheduling and service discovery
Auto-Scaling	Kubernetes HPA	Horizontal Pod Autoscaler
Log Aggregation	Logstash	Log parsing and routing
Search & Indexing	Elasticsearch	Centralised log storage
Monitoring	Kibana	Real-time log dashboards
Tunnel/Webhook	ngrok	Expose local Jenkins to GitHub webhooks

3. Application Architecture

3.1 Service Breakdown

Each microservice is independently deployable with its own Docker image, Jenkins pipeline, and Kubernetes deployment:

Service	Port (container → host)	Responsibilities	Docker Image
api-gateway	8080 → 8202	JWT validation, request routing to downstream services	3piradians/api-gateway
auth-service	8081 → 8200	User registration, login, logout, JWT issuance	3piradians/auth-service
app-service	8082 → 8201	Portfolio, holdings, market data, admin dashboard	3piradians/app-service
MySQL	3306	Shared database for all services	mysql:8.0

3.2 Source Code Structure

The project is organised into three independent Spring Boot projects, each with its own package structure:

auth-service/src/main/java/com/stockportfolio/auth/

- config/ — SecurityConfig, AppConfig
- controller/ — AuthController (/auth/register, /auth/login, /auth/logout)
- service/ — AuthService (BCrypt password hashing, JWT issuance)
- security/ — JwtUtil, JwtAuthFilter, CustomUserDetailsService
- entity/ — User.java | repository/ — UserRepository

api-gateway/src/main/java/com/stockportfolio/gateway/

- filter/ — JwtAuthFilter (GlobalFilter — validates JWT, passes to downstream)
- config/ — GatewayConfig, route definitions

src/main/java/com/stockportfolio/app/ (app-service)

- controller/ — PortfolioController, HoldingController, MarketController, AdminController
- service/ — PortfolioService, HoldingService, MarketService
- entity/ — Portfolio, Holding, Transaction | repository/ — JPA repositories

3.3 Data Model (ERD)

A single MySQL database (stockportfolio) is shared with logical table ownership: auth-service owns the users table; app-service owns portfolios, holdings, and transactions.

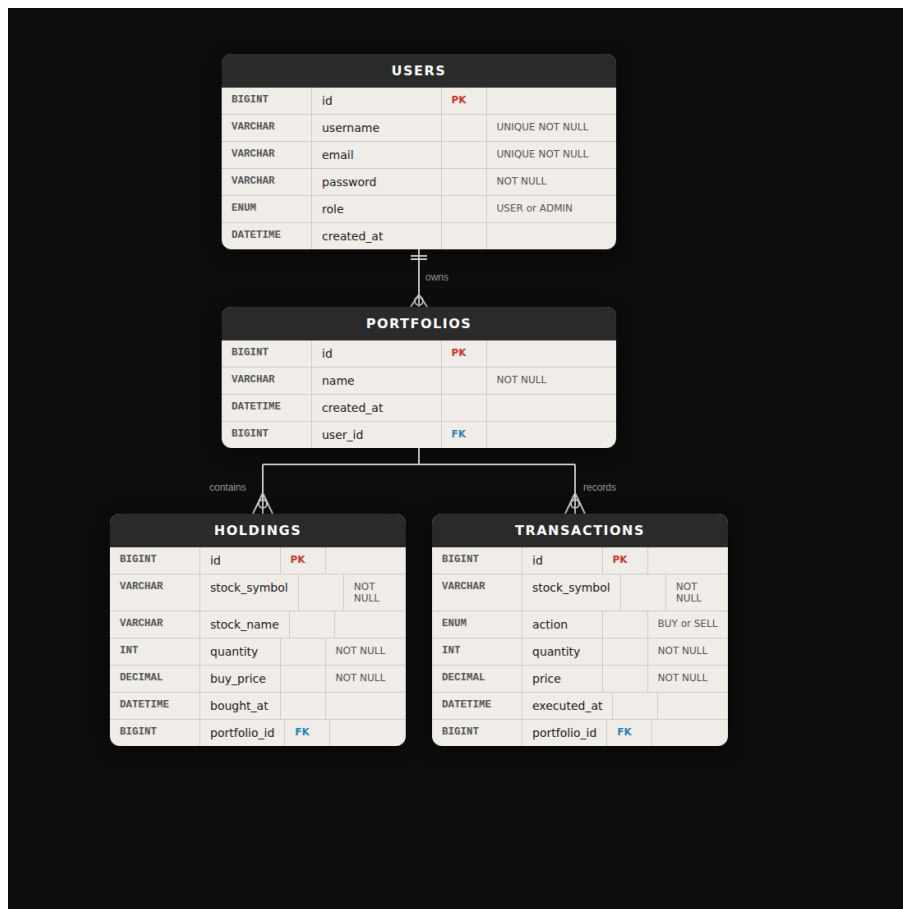


Figure 2: Entity Relationship Diagram

3.4 REST API Endpoints

Method	Endpoint	Description	Access
POST	/auth/register	Register new user	Public
POST	/auth/login	Login; issues JWT cookie	Public
GET	/auth/logout	Clear JWT cookie	Authenticated
GET	/portfolio	View all portfolios	User
POST	/portfolio/create	Create new portfolio	User
GET	/portfolio/{id}/holdings	Holdings + live P&L	User
POST	/portfolio/{id}/holdings/add	Buy a stock	User
POST	/portfolio/{id}/holdings/remove/{hid}	Sell a stock	User
GET	/market/search?symbol=AAPL	Fetch live stock price	User
GET	/admin	Admin dashboard	Admin only

4. Security Design

4.1 Authentication — Stateless JWT

1. User submits credentials via /auth/login
2. AuthService validates credentials and verifies the BCrypt-hashed password
3. A signed JWT token is generated by JwtUtil using the JWT_SECRET environment variable
4. The token is stored in an HTTP-only cookie (not accessible to JavaScript — prevents XSS token theft)
5. api-gateway validates the JWT cookie on every request before forwarding downstream
6. app-service also validates the JWT to ensure requests did not bypass the gateway

4.2 Role-Based Access Control

- USER role: Access to all portfolio, holdings, and market endpoints
- ADMIN role: Additional access to the /admin dashboard

4.3 Secrets Management

Secret	How Protected
DB_USERNAME / DB_PASSWORD	Stored in .env (gitignored); injected as container env vars via Ansible
JWT_SECRET	Shared across all 3 services; never hardcoded in source
FINNHUB_API_KEY	Injected at runtime to app-service only
CI/CD Credentials	Stored as Jenkins Credentials (not in Jenkinsfile)
Production Secrets	Encrypted with Ansible Vault (ansible/vars/secrets.yml)
Docker Hub Credentials	Stored as Jenkins usernamePassword credentials

5. DevOps Pipeline — Detailed Breakdown

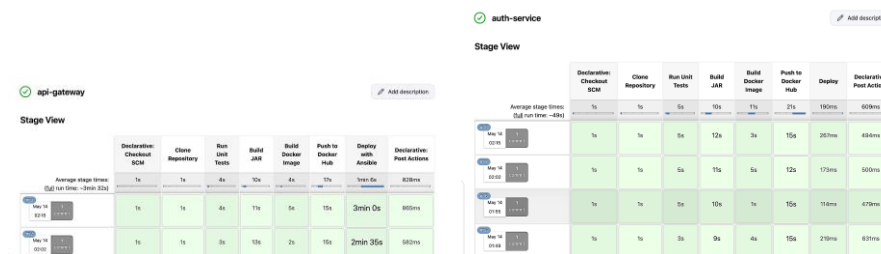
5.1 Version Control Strategy

Branch	Owner	Purpose
main	Both	Production-ready, protected, triggers all Jenkins pipelines
feature/auth	Chaitanya	Authentication module
feature/portfolio	Chaitanya	Portfolio management
feature/market-data	Chaitanya	Finnhub API integration
feature/jenkins	Chaitanya	Jenkins CI/CD pipeline
feature/kubernetes	Chaitanya	Kubernetes deployment configs
feature/docker	Pankaj	Dockerfile + Docker Compose
feature/ansible	Pankaj	Ansible playbooks + Vault
feature/elk	Pankaj	ELK Stack + Kibana dashboards

5.2 Continuous Integration — Jenkins (4 Pipelines)

Pipeline Job	Jenkinsfile	Stages	Deploys?
api-gateway	Jenkinsfile	Clone → Test → Build JAR → Docker Build → Push → Ansible Deploy	YES — full stack
auth-service	Jenkinsfile.auth	Clone → Test → Build JAR → Docker Build → Push	NO — image only
app-service	Jenkinsfile.app	Clone → Test → Build JAR → Docker Build → Push	NO — image only
stock-portfolio-manager	Jenkinsfile.app	Clone → Test → Build JAR → Docker Build → Push	NO — image only

Figure 3: Jenkins Pipeline Stage View — All 4 Pipelines



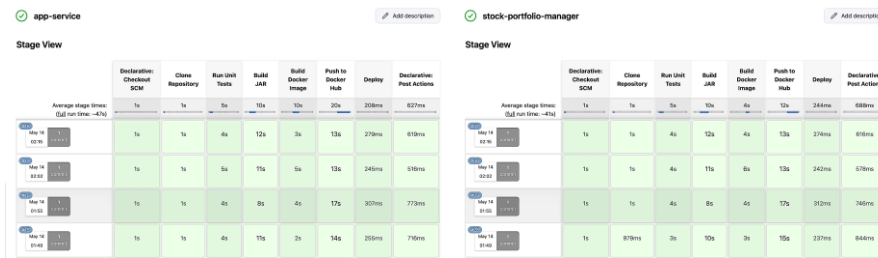


Figure 3: api-gateway (top-left) · auth-service (top-right) · app-service (bottom-left) · stock-portfolio-manager (bottom-right)

5.3 Containerisation — Docker

- Stage 1 (Build): maven:3.9.4-eclipse-temurin-21 — compiles and packages the JAR
- Stage 2 (Run): eclipse-temurin:21-jre-alpine — minimal JRE-only image (~180MB vs ~600MB)

Docker Compose Stack (7 Services):

Service	Image	Port	Role
mysql	mysql:8.0	3306	Primary database with health check
auth-service	3piradians/auth-service:latest	8081 → 8200	Authentication microservice
app-service	3piradians/app-service:latest	8082 → 8201	Business logic microservice
api-gateway	3piradians/api-gateway:latest	8080 → 8202	API gateway and JWT validator
elasticsearch	elasticsearch:8.13.0	9200	Log search and storage
logstash	logstash:8.13.0	5044/5000	Log ingestion and parsing
kibana	kibana:8.13.0	5601	Log visualisation

5.4 Configuration Management — Ansible (4 Roles)

Role	Responsibilities
mysql	Force-removes old containers, creates Docker network, starts MySQL with health check
app_server	Pulls latest images, starts auth-service, app-service, and api-gateway containers
elk_stack	Starts Elasticsearch, Logstash (with 3-service log config), and Kibana
kubernetes	Applies all K8s manifests in order: ConfigMap → Secrets → Deployments → HPA → Ingress → ELK → Filebeat

5.5 Container Orchestration — Kubernetes with HPA

Manifest	Resource	Description
configmap.yaml	ConfigMap: stockportfolio-config	DB_HOST, DB_PORT, DB_NAME, service URLs
secrets.yaml	Secret: stockportfolio- secrets	DB credentials, JWT_SECRET, FINNHUB_API_KEY (base64)
auth-service- deployment.yaml	Deployment + Service	auth-service: 2 replicas, probes, NodePort 30081
app-service- deployment.yaml	Deployment + Service	app-service: 2 replicas, probes, NodePort 30082
api-gateway- deployment.yaml	Deployment + Service	api-gateway: 2 replicas, probes, NodePort 30080
hpa.yaml	HPA x3	min:2 / max:5 / CPU:70% for all 3 services
ingress.yaml	Ingress (nginx)	/ → api-gateway:8080, /kibana → kibana:5601
elk-stack.yaml	ELK Deployments	Elasticsearch, Logstash, Kibana on Kubernetes
filebeat.yaml	DaemonSet	Filebeat on all nodes, collects from /var/log/pods

5.6 Monitoring — ELK Stack

Log Flow: Spring Boot (logback) → /app/logs/ → Docker Volume → Logstash reads → Grok filter parses → Elasticsearch indexes → Kibana dashboards visualise

Logstash Grok Pattern:

```
%{TIMESTAMP_ISO8601} [%{DATA:thread}] %{LOGLEVEL:level} %{DATA:logger} -
%{GREEDYDATA:log_message}
```

```
{
  "stockportfolio-logs-2026.05.13": {
    "aliases": {},
    "mappings": {
      "properties": {
        "@timestamp": {
          "type": "date",
          "version": {
            "type": "text",
            "fields": {
              "keyword": {
                "type": "keyword",
                "ignore_above": 256
              }
            }
          },
          "event": {
            "properties": {
              "original": {
                "type": "text",
                "fields": {
                  "keyword": {
                    "type": "keyword",
                    "ignore_above": 256
                  }
                }
              },
              "name": {
                "type": "text",
                "fields": {
                  "keyword": {
                    "type": "keyword",
                    "ignore_above": 256
                  }
                }
              },
              "log": {
                "properties": {
                  "path": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "log_message": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "logger": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "message": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "service": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "tags": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "thread": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  }
                }
              },
              "tier_preference": {
                "data_content": {
                  "number_of_shards": "1",
                  "provided_name": "stockportfolio-logs-2026.05.13",
                  "creation_date": "17788489472",
                  "number_of_replicas": "1",
                  "uid": "200x3Kt0y0X1CtJ8map",
                  "version": {
                    "created": "8503000"
                  }
                }
              }
            }
          }
        }
      }
    }
  },
  "stockportfolio-logs-2026.05.14": {
    "aliases": {},
    "mappings": {
      "properties": {
        "@timestamp": {
          "type": "date",
          "version": {
            "type": "text",
            "fields": {
              "keyword": {
                "type": "keyword",
                "ignore_above": 256
              }
            }
          },
          "event": {
            "properties": {
              "original": {
                "type": "text",
                "fields": {
                  "keyword": {
                    "type": "keyword",
                    "ignore_above": 256
                  }
                }
              },
              "name": {
                "type": "text",
                "fields": {
                  "keyword": {
                    "type": "keyword",
                    "ignore_above": 256
                  }
                }
              },
              "log": {
                "properties": {
                  "path": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "log_message": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "logger": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "message": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "service": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "tags": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  },
                  "thread": {
                    "type": "text",
                    "fields": {
                      "keyword": {
                        "type": "keyword",
                        "ignore_above": 256
                      }
                    }
                  }
                }
              },
              "tier_preference": {
                "data_content": {
                  "number_of_shards": "1",
                  "provided_name": "stockportfolio-logs-2026.05.14",
                  "creation_date": "17788558232",
                  "number_of_replicas": "1",
                  "uid": "200x3Kt0y0X1CtJ8map",
                  "version": {
                    "created": "8503000"
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Figure 4: Elasticsearch — Indices for stockportfolio-logs

6. End-to-End Pipeline Flow

- Developer pushes code to GitHub (git push origin main)
- GitHub Webhook fires to Jenkins via ngrok tunnel
- All 4 Jenkins pipelines trigger simultaneously
- auth-service pipeline: builds JAR → Docker image → pushes 3piradians/auth-service:latest
- app-service pipeline: builds JAR → Docker image → pushes 3piradians/app-service:latest
- api-gateway pipeline: builds JAR → Docker image → pushes 3piradians/api-gateway:BUILD_NUMBER
- api-gateway pipeline triggers Ansible deploy playbook
- Ansible mysql role: removes old containers, starts fresh MySQL
- Ansible app_server role: pulls images, starts auth-service → app-service → api-gateway
- Ansible elk_stack role: starts ELK stack for log monitoring
- Ansible kubernetes role: applies all K8s manifests (ConfigMap, Secrets, Deployments, HPA, Ingress)
- Application is live with all microservices running
- Logs flow from all 3 services through Logstash to Elasticsearch, visible in Kibana

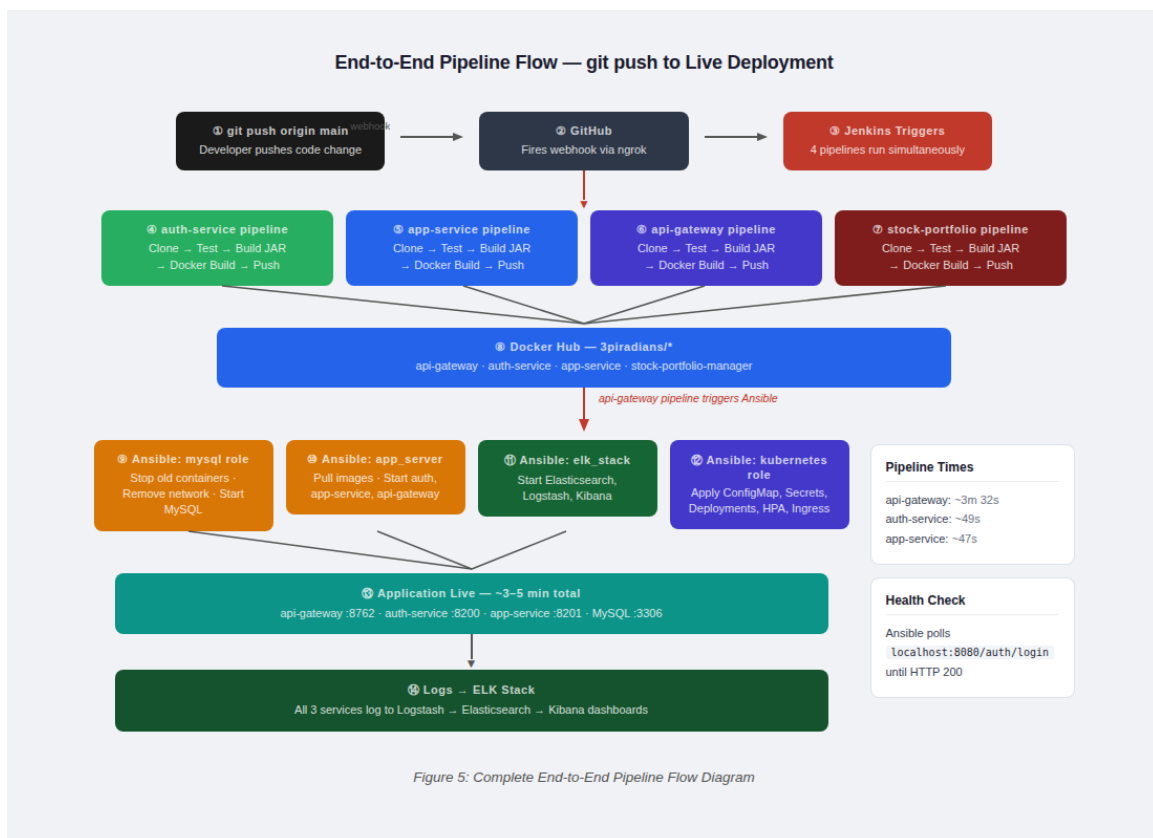
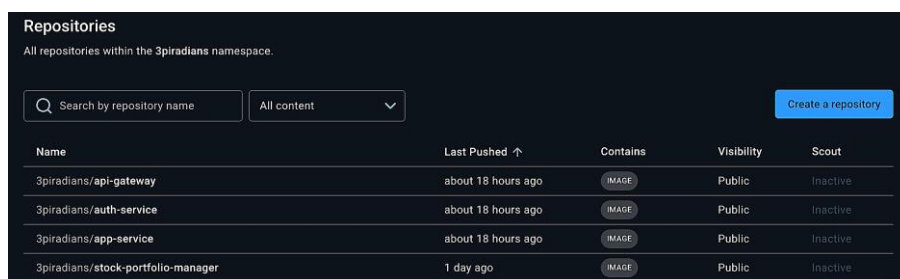


Figure 5: Complete End-to-End Pipeline Flow Diagram

7. Docker Hub Images

Image	Tags	Description
3piradians/api-gateway	:latest, :BUILD_NUMBER	Spring Cloud Gateway — JWT validation and routing
3piradians/auth-service	:latest	Authentication microservice — JWT issuance
3piradians/app-service	:latest	Business logic microservice — Portfolio/Holdings/Market
3piradians/stock-portfolio-manager	:latest, :BUILD_NUMBER	app-service build (stock-portfolio-manager job)



The screenshot shows the Docker Hub interface for the 3piradians namespace. It features a search bar, a dropdown for content type (set to 'All content'), and a 'Create a repository' button. Below is a table listing four repositories: 3piradians/api-gateway, 3piradians/auth-service, 3piradians/app-service, and 3piradians/stock-portfolio-manager. Each entry shows the last push time, a status icon, visibility (Public), and a Scout link.

Repositories				
All repositories within the 3piradians namespace.				
Q Search by repository name		All content	Create a repository	
Name	Last Pushed ↑	Contains	Visibility	Scout
3piradians/api-gateway	about 18 hours ago	IMAGE	Public	Inactive
3piradians/auth-service	about 18 hours ago	IMAGE	Public	Inactive
3piradians/app-service	about 18 hours ago	IMAGE	Public	Inactive
3piradians/stock-portfolio-manager	1 day ago	IMAGE	Public	Inactive

Figure 6: Docker Hub — All 4 Images in 3piradians Namespace

8. Advanced Features

8.1 Ansible Vault for Secrets

All sensitive credentials (database passwords, JWT secret, Finnhub API key) are encrypted using Ansible Vault in `ansible/vars/secrets.yml`. The vault password is never stored in the repository and is provided only at pipeline runtime via Jenkins credentials.

8.2 Kubernetes HPA

- `minReplicas`: 2 — Always 2 pods for High Availability
- `maxReplicas`: 5 — Scale to 5 pods under load
- `CPU threshold`: 70% — Trigger scale-out when average CPU exceeds 70%

8.3 Microservices Architecture

- Service isolation — `auth-service`, `app-service`, and `api-gateway` are independently deployable
- Gateway pattern — JWT validation centralised at `api-gateway` level
- Independent CI/CD — 4 separate Jenkins pipelines, each owning their service
- Shared database with logical table ownership

8.4 Ansible Roles — Modular Design

Ansible configuration is organised into 4 reusable roles (`mysql`, `app_server`, `elk_stack`, `kubernetes`). This modular design allows individual roles to be run independently for targeted deployments.

8.5 Filebeat DaemonSet on Kubernetes

A Filebeat DaemonSet runs on all Kubernetes nodes, collecting pod logs from `/var/log/pods` with Kubernetes metadata enrichment (namespace, pod name, container). This provides complete observability across all microservices and infrastructure pods.